

JC4004 – Computational Intelligence

Sessions 5 & 6: Reinforcement Learning - An Introduction of Concepts

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (yongchao.huang@abdn.ac.uk)

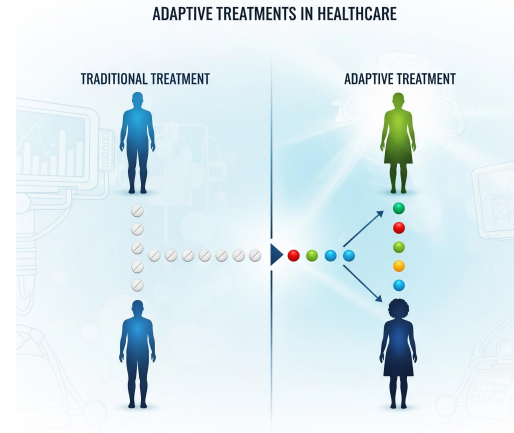
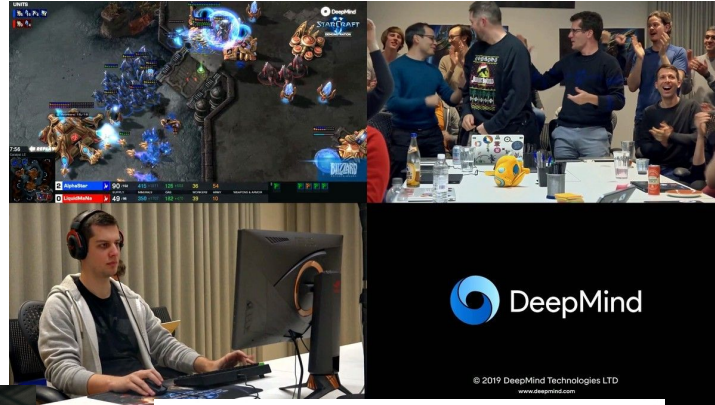
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)s

Oct-Dec 2025

Background

- It's about decision making under uncertainty.
- RL allows autonomous learning without labeled data.
- Applications:
 - Self-driving cars learning to navigate safely.
 - Game-playing AI (AlphaGo, AlphaStar).
 - Robots learning to walk, grasp, fly.
 - Adaptive treatments in healthcare.
 - ...

Real-World Examples

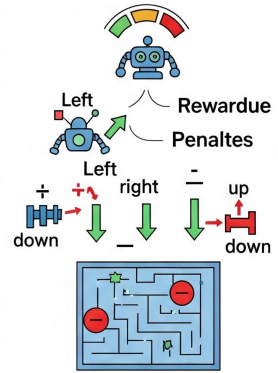
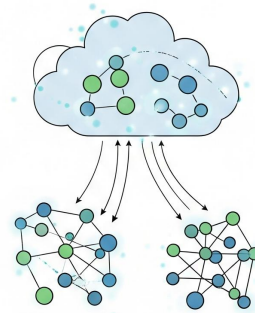
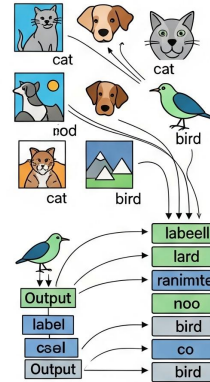


Real-World Examples

- Games: AlphaGo, AlphaZero, Dota2 bots.
- Robotics: Quadruped walking, drone control.
- Recommendation Systems: Netflix, YouTube personalization.
- Healthcare: Adaptive treatment strategies.

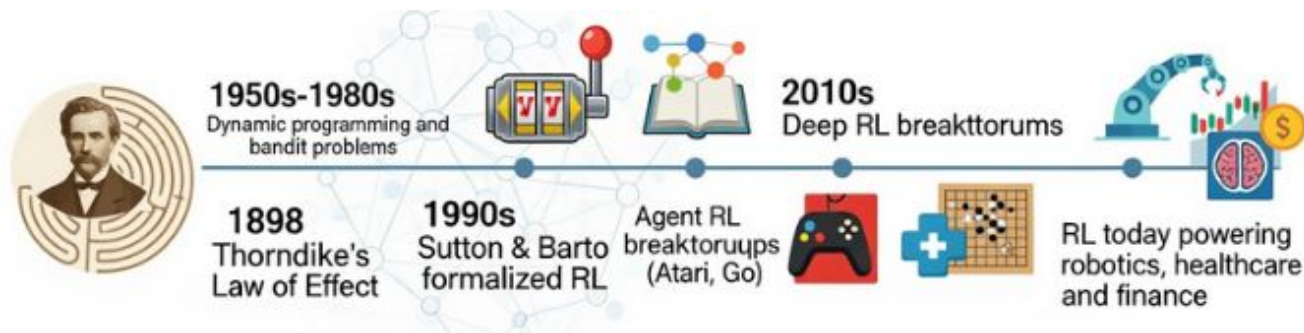
AI Paradigms

- Supervised Learning: Input $\rightarrow$ Output mapping with labels.
- Unsupervised Learning: Discover patterns without labels.
- Reinforcement Learning: Learn actions to maximize (cumulative) reward.
RL = trial-and-error with delayed feedback.



RL: history

- 1898: Thorndike's Law of Effect – behavior shaped by rewards/punishments.
- 1950s–1980s: Dynamic programming, bandit problems.
- 1990s: Sutton & Barto formalized RL.
- 2010s: Deep RL breakthroughs (Atari, Go).
- RL today powers robotics, healthcare, finance, etc ...



RL: Key Idea

- RL is about learning from interaction.
- Agent chooses actions → Environment responds → Agent updates.
- Goal: maximize expected long-term reward, not just immediate reward.
- Distinguishing feature: delayed consequences matter.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

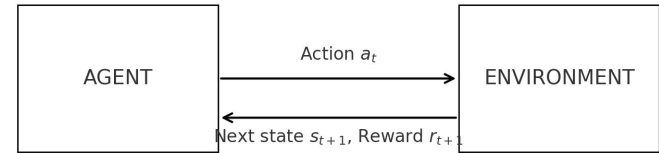
RL: Foundations

Agent-Environment Loop

Core cycle:

- Agent observes **state** s_t .
- Takes **action** a_t .
- Gets **reward** r_{t+1} and next state s_{t+1} .

Loop repeats until task ends.



Time $t \rightarrow t+1 \rightarrow t+2 \dots$

Reward

- Reward is a **scalar feedback** indicating success/failure.
- Can be immediate (chess win/loss at end) or sparse (robot reaches goal).
- Challenges: designing good reward functions is hard (“reward hacking”).

$$\max_{\pi} \mathbb{E}_{\pi} [G_t]$$

Markov Decision Processes (MDPs)

RL is formalized as an **MDP**:

- **S**: set of states.
- **A**: set of actions.
- **P(s'|s,a)**: transition probabilities.
- **R(s,a)**: expected reward.
- **γ** : discount factor.

Assumption: **Markov property** – next state depends only on current state + action.

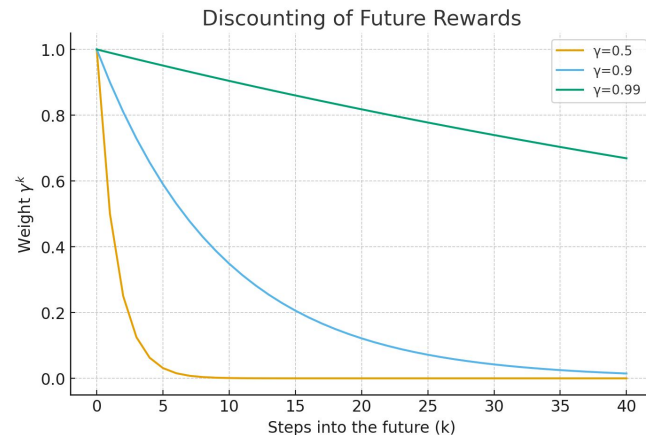
Discounted Return

The return at time t :

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad 0 \leq \gamma < 1$$

Why discount?

- Immediate rewards count more than distant ones.
- Ensures convergence of infinite sum.



Policy

Defines how an agent chooses actions.

- Deterministic: $a=\pi(s)$
- Stochastic: $\pi(a | s)$

Policies are the **core object of learning** in RL.

Value Functions

State-value:

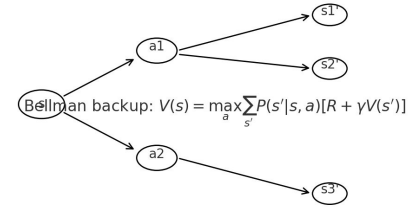
$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Action-value (Q-function):

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

These quantify “how good” a state or action is under a policy.

Bellman Equations



- Recursive definition of value functions.
- Expresses value of a state in terms of successor states.
- Foundation of dynamic programming and most RL algorithms.

$$V^\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]$$

The value of state s = immediate reward + discounted value of next states, weighted by probabilities and policy.

For policy π :

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Optimality Principle

Optimal policy π^* maximizes value:

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

Bellman optimality equations define V^*, Q^* .

Goal: find π^* .

RL Algorithms Overview

Classification of RL Methods

- Value-based: learn value functions (Q-learning, SARSA).
- Policy-based: directly optimize policies (REINFORCE).
- Actor–Critic: combine both.
- Model-based: learn environment model and plan.

Classification of RL Methods

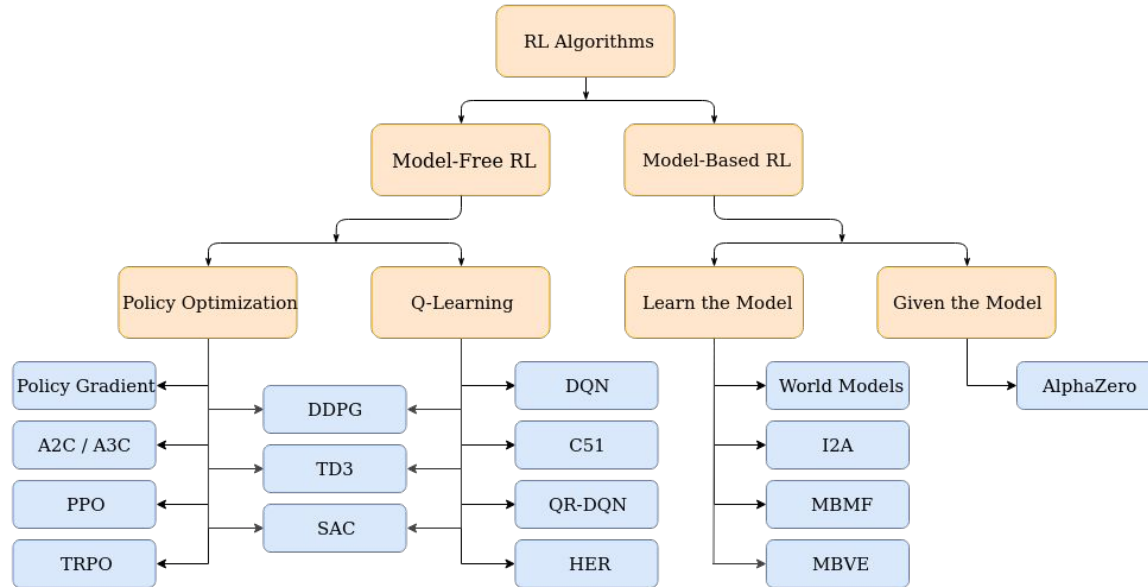


image source: https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

Classification of RL Methods

Method	Core Idea	Pros	Cons
Value-based	Learn value functions (e.g., Q-learning, SARSA).	Simple to implement; well-studied; works well in discrete action spaces.	Difficult with continuous actions; can be unstable with function approximation.
Policy-based	Directly optimize policy parameters (e.g., REINFORCE).	Works with continuous actions; can represent stochastic policies; often better exploration.	High variance; may converge slowly; sensitive to learning rates.
Actor–Critic	Combine policy learning (actor) and value learning (critic).	Reduces variance vs pure policy gradient; more stable training.	More complex; tuning actor–critic balance is tricky.
Model-based	Learn an environment model and plan ahead.	Sample-efficient; enables planning and imagination-based learning.	Requires accurate models; harder to scale to high-dimensional environments.

Dynamic Programming (DP)

- Requires full knowledge of environment dynamics.
- Policy evaluation $\rightarrow$ Policy improvement.
- Policy iteration vs Value iteration.
- Not practical for large problems but foundational.

Monte Carlo (MC) Methods

- Learn from complete episodes.
- Estimate value by averaging returns.
- No need for transition probabilities.
- Cons: can't be used in non-episodic tasks.

Temporal Difference (TD) Learning

- Bootstrapping: update based on estimates rather than full returns.
- Combines advantages of MC and DP.

- **TD(0) Update Rule:**

$$V(s) \leftarrow V(s) + \alpha(r + \gamma V(s') - V(s))$$

- **TD Error (δ):**

$$\delta_t = r + \gamma V(s') - V(s)$$

Update is $V(s) \leftarrow V(s) + \alpha \delta_t$.

Temporal Difference (TD) Learning

$$V(s) \leftarrow V(s) + \alpha(r + \gamma V(s') - V(s))$$

- Key idea: Update value estimates based on one-step predictions (bootstrapping) rather than waiting for full episode returns.
- TD Error: The difference between predicted value and improved estimate from reward + next state value.
- Advantages:
 - Online & incremental (learns at each step, not only after episode ends).
 - Combines strengths of Monte Carlo (MC) (learn from real experience) and Dynamic Programming (DP) (bootstrapping).
 - Works without knowing the environment model.

Q-Learning (Off-policy)

- Learns optimal action-value function regardless of behavior policy.
- Simple and widely used.

Update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Q-Learning (Off-policy): Example

Suppose we have a **Gridworld** with 4 actions (Up, Down, Left, Right).

The Q table on the right side:

- Each row corresponds to a state.
- Each column is the estimated value of taking that action in that state.
- Q-learning update adjusts these numbers step by step, pushing them toward the optimal action-values.
- Eventually, the max entry per row shows the best action (the learned policy).

State	Up	Down	Left	Right
S1	0.0	0.5	0.2	0.1
S2	-0.1	0.0	0.4	0.6
S3	0.3	-0.2	0.0	0.8
S4 (goal)	0.0	0.0	0.0	0.0

SARSA (On-policy)

- SARSA stands for *State–Action–Reward–State–Action*.
- On-policy: updates are based on the action actually taken by the current policy, not the greedy max action.
- This makes SARSA more conservative and less likely to diverge in risky environments.
- Often safer in domains with stochastic dynamics or high risk (e.g., driving a car near a cliff in Gridworld).

SARSA (On-policy)

Update Rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a))$$

Compare with Q-learning: SARSA uses $Q(s', a')$ (actual action), whereas Q-learning uses $\max_{a'} Q(s', a')$.

- Uses action actually taken, not max action, i.e. update follows the real experience path (on-policy);
- More conservative but safer.

Advanced RL Approaches

Policy Gradient Methods

Instead of learning a value function, we directly optimize the policy parameters θ .

Objective: maximize expected return

$$J(\theta) = \mathbb{E}_{\pi}[G_t]$$

Gradient (Policy Gradient Theorem):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) G_t \right]$$

Policy Gradient Methods

- Gradient ascent on policy parameters.
- *Intuition*: increase probability of actions that lead to high returns.
- Suitable for continuous action spaces where Q-learning struggles.
- REINFORCE algorithm.

Policy Gradient Methods

- REINFORCE (pseudo-code)

```
Initialize policy parameters  $\theta$  randomly
Repeat (for each episode):
    Generate an episode using policy  $\pi_\theta$ 
    For each time step  $t$  in the episode:
        Compute return  $G_t$  from  $t$  onward
        Update policy parameters:
             $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t | s_t) G_t$ 
```

- REINFORCE is **Monte Carlo**: requires full episodes.
- Updates have **high variance**, but unbiased.
- Can be improved with a **baseline** (e.g., subtract value function estimate).

Actor–Critic Methods

Actor: parameterized policy $\pi_{\theta}(a|s)$ that selects actions.

Critic: estimates value function $V_w(s)$ (or $Q_w(s, a)$) to judge the actor's decisions.

Update loop:

- Critic computes a **TD error**:

$$\delta_t = r + \gamma V_w(s') - V_w(s)$$

- Actor updates policy parameters:

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \log \pi_{\theta}(a|s)$$

- Critic updates its value function parameters.
- **Advantage:** Reduces variance of policy gradient estimates compared to pure REINFORCE.
- **Used in modern deep RL algorithms:** A2C, A3C, PPO, DDPG.

Actor–Critic Methods

Actor: updates policy.

Critic: evaluates policy via value function.

The actor learns what to do, the critic learns how good it was.

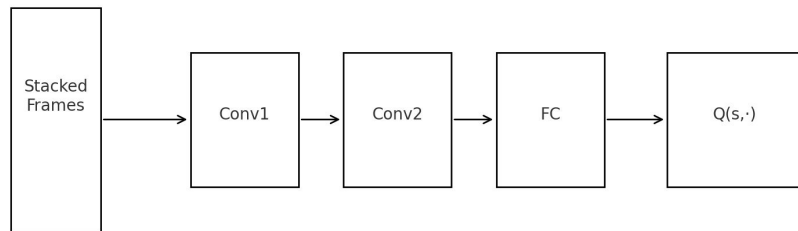
Lower variance than pure policy gradient.

Deep RL (DQN)

Combine Q-learning with deep neural nets.

Breakthrough: play Atari games directly from pixels.

Challenges: instability, overestimation bias.

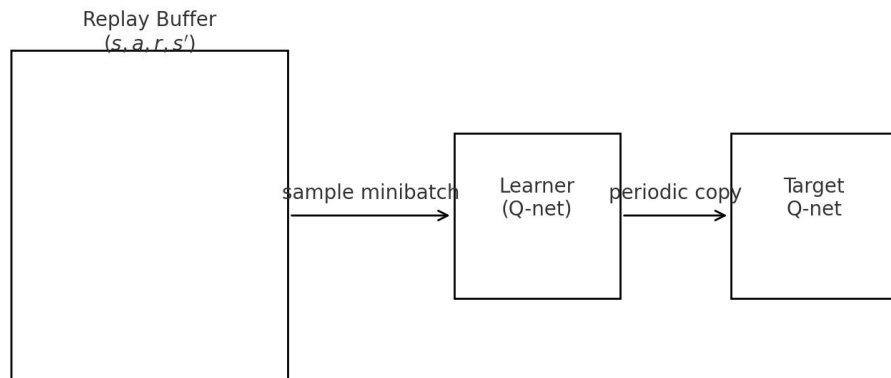


Deep Q-Network (DQN): CNN → FC → Q-values

Experience Replay & Target Networks

Replay buffer: break correlations, improve efficiency.

Target network: stabilize learning by holding fixed parameters temporarily.



Model-Based RL

Learn environment model.

Use model to simulate and plan.

More sample-efficient, but harder to model complex environments.

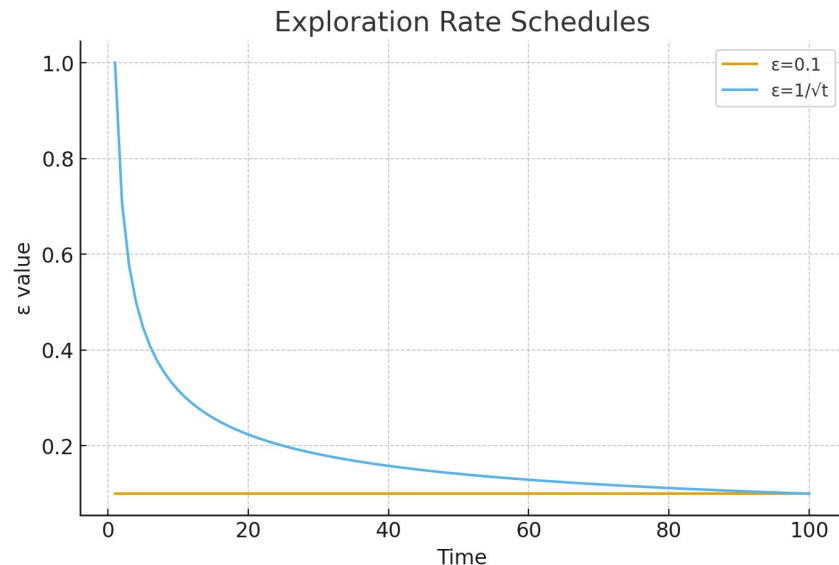
Exploration vs Exploitation

Exploration: discover new strategies.

Exploitation: use known strategies.

Methods:

- ϵ -greedy.
- Upper Confidence Bound (UCB).
- Thompson sampling.



Pros, Cons, Challenges

Pros of RL

- Learns from interaction, no labels needed.
- Can handle sequential decision-making.
- Flexible and general.

Cons of RL

- Requires many samples (sample inefficient).
- Training often unstable.
- Sensitive to reward design.

Comparison

- Value-based: simple, but unstable with function approximation.
- Policy-based: handles continuous actions, but high variance.
- Actor–Critic: balance, but complex.
- Model-based: efficient, but requires accurate models.

Challenges

- Credit assignment problem.
- Sparse rewards.
- Exploration trade-offs.
- Scalability to real-world tasks.

Challenges

- **Credit assignment problem.**

In RL, rewards are often delayed. An agent may take a long sequence of actions before receiving a final reward or penalty.

The challenge: which action(s) in the past deserve the credit (or blame) for that outcome?

e.g. A robot explores a maze for 50 steps and finally reaches the goal (+1 reward).

Did the reward come mainly because of the first turn it took, or the last few moves?

Without a mechanism like value functions, eligibility traces, or backpropagation through time, the agent cannot know how to assign responsibility.

Challenges

- **Sparse Rewards in RL**

In many real-world tasks, useful rewards are rare. The agent performs many actions but only occasionally gets feedback (e.g., win/lose at the end of a game, or reaching a goal after thousands of steps).

This makes learning difficult and slow, because the agent gets very little guidance about whether its intermediate actions are good or bad.

e.g. Maze navigation: The agent only gets a +1 reward at the end if it reaches the goal; all other steps give 0. Without intermediate signals, the agent has to explore blindly until it stumbles upon the goal.

Applications

Games & Simulations

- *Why games?*
 - a. Provide a controlled environment with clear rules and rewards.
 - b. Allow safe, repeatable experiments (no real-world risks).
- **Classic successes of RL in games:**
 - a. Chess: Deep Blue (classical AI) → AlphaZero (RL + self-play).
 - b. Go: AlphaGo defeated world champion (2016).
 - c. Atari 2600: Deep Q-Network (DQN) learned directly from pixels.
 - d. Dota2 / StarCraft II: multi-agent RL, long-horizon strategies.
- *Why simulations?*
 - a. Let agents train for millions of episodes quickly.
 - b. Cheaper and faster than real-world robotics.
- Games are widely used to compare RL algorithms.
- Progress in games often translates to progress in robotics, healthcare, and other domains.

Robotics

- Navigation (drones, rovers).
- Manipulation (grasping, assembling).
- RL enables learning control without explicit programming.

Healthcare

- Adaptive drug dosing.
- Personalized treatments.
- Caution: safety and ethics critical.

Finance & Economics

- Portfolio optimization.
- Algorithmic trading.
- Risk-sensitive RL approaches.

Recommendation Systems

- **Personalized content feeds** (e.g., YouTube, Netflix, TikTok).
- **RL adapts to user interactions** (clicks, likes, watch time).

e.g. YouTube's recommendation system uses multi-armed bandits and deep RL to decide which video to show next.

- a. The state: user's profile, history, and current context.
- b. The action: which video to recommend.
- c. The reward: whether the user clicks, watches, or skips.

Over time, RL learns to maximize long-term engagement rather than just short-term clicks.

RL for LLMs

- **RLHF = Reinforcement Learning with Human Feedback**
 - a. Used to fine-tune large language models such as **ChatGPT**.
 - b. Aligns model outputs with **human preferences and values**.
- **Pipeline:**
 - a. **Pre-training:** LLM learns next-token prediction on massive text datasets.
 - b. **Supervised fine-tuning:** train on human-written responses.
 - c. **RLHF stage:**
 - i. A **reward model** is trained from human rankings of responses.
 - ii. The LLM acts as the **policy**.
 - iii. Using **PPO (Proximal Policy Optimization)**, the policy is updated to maximize the reward model's score.
- **RLHF** makes LLMs safer, more helpful, and less toxic. Still faces challenges: bias in feedback, reward hacking, scalability.

Examples

Gridworld

Setup:

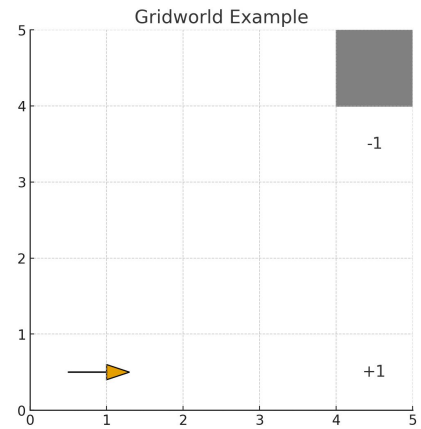
- a. The agent starts in a grid (bottom-left).
- b. Some cells are **terminal states** with rewards (+1 goal, -1 trap).
- c. Empty cells give reward 0.

Actions:

- d. The agent can move **up, down, left, right**.
- e. Sometimes transitions are **stochastic** (e.g., slip into the wrong cell with 10% probability).

Objective:

- f. Maximize cumulative reward by reaching the goal (+1) while avoiding the trap (-1).
- g. Learn an **optimal policy** (a map from states $\rightarrow$ best actions).



grid with arrows in each cell (the learned optimal policy).
Agent learns the shortest safe path to the +1 goal.

Gridworld

- **Learning method:**
 - a. Use **Value Iteration** or **Policy Iteration**.
 - b. Example:
 - i. Compute value for each state.
 - ii. Derive policy: always move towards the goal unless risk of falling into trap is high.
- Small MDP with states, rewards, transitions.
- Apply value iteration to compute optimal policy.

Multi-Armed Bandits

- Imagine a row of slot machines (“one-armed bandits”).
- Each machine gives a reward with some unknown probability distribution.
- You have limited pulls: which machines should you play, and how often, to maximize total reward?

Core challenge:

- Exploration: trying different arms to learn their reward probabilities.
- Exploitation: playing the arm that seems best so far.
- Need to balance both to avoid missing better options.

Multi-Armed Bandits

Performance measure:

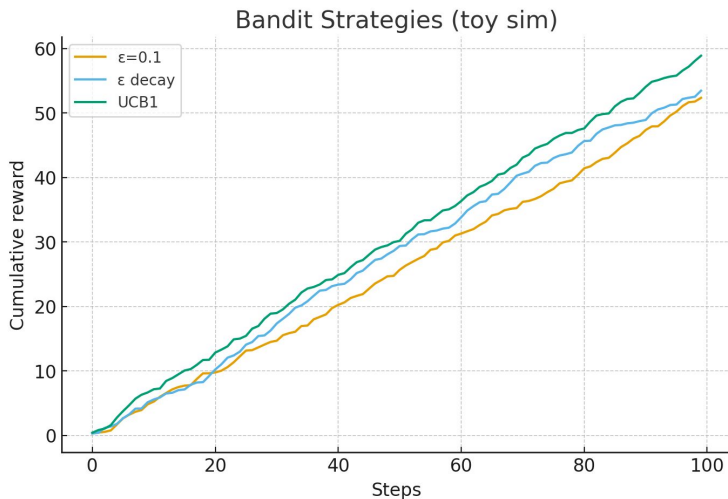
- *Regret*: the difference between the reward you actually got and the reward you would have gotten if you always picked the best arm.
- *Goal*: design strategies to minimize regret over time.

Popular strategies:

- *ϵ -greedy*: explore randomly with probability ϵ .
- *UCB* (Upper Confidence Bound): pick the arm with the best balance of average reward and uncertainty.
- *Thompson Sampling*: Bayesian sampling approach.

Multi-Armed Bandits

- Trade-off between exploration and exploitation.
- Regret minimization.



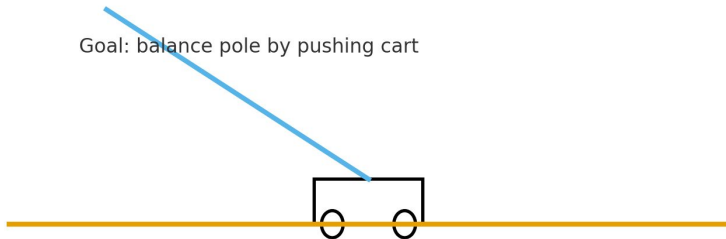
CartPole

- A classic RL benchmark from control theory.

A cart moves left or right on a track while balancing a pole attached to it. The agent must apply forces to the cart to keep the pole upright for as long as possible. The task ends if the pole falls too far or the cart goes off track.

- Widely used as a simple testbed for RL algorithms (e.g. DQN, policy gradients).

Goal: balance pole by pushing cart



CartPole

- Classic control problem: keep pole balanced.
- DQN solution: neural network approximates Q-values.

Summary

- RL = learning from interactions.
- Core concepts: MDPs, policies, value functions, Bellman equations.
- Methods: DP, MC, TD, Q-learning, Policy Gradient, Deep RL.
- Applications across domains.

Future Directions

- Safe & explainable RL.
- Multi-agent RL.
- Transfer and meta-RL.
- Scaling RL for real-world impact.

Textbooks / References

- Richard S. Sutton and Andrew G. Barto (2018), *Reinforcement Learning: An Introduction* (2nd Edition). MIT Press.
- Csaba Szepesvári (2009), *Algorithms for Reinforcement Learning*. Morgan & Claypool.

Questions and discussion